

1. Motivation and Selection Criteria

As we navigate the vast open-source codebase of OpenRefine, identifying where to contribute requires a strategic approach rather than random selection. Based on the concept of "Social Context" and the importance of understanding the domain and design philosophy, I used the following three criteria to identify my target issues:

Domain Continuity: Issues related to the features I recently analyzed (such as Clustering and Project management).

Appropriate Scope: Issues labeled as good first issue or those with clear, localized reproduction steps.

Community Engagement: Issues that feature active discussions or clear guidance from core maintainers.

Why these criteria are useful at this time: At this stage of the project, our primary goal is to transition from abstract architectural modeling to concrete code reading. By selecting issues within a domain we already conceptually understand (Criterion 1), we avoid being overwhelmed by unrelated system complexities. Furthermore, prioritizing appropriately scoped issues with community guidance (Criteria 2 & 3) allows us to observe the "social context" in action—how maintainers discuss the system's philosophy and guide newcomers.

2. Identified Issues

Issue 1: Integrate PassJoin cluster library

Link: <https://github.com/OpenRefine/OpenRefine/issues/983>

Discussion: This feature request proposes integrating a new, high-performance clustering algorithm called PassJoin, which is specifically optimized for edit-distance clustering. Unlike the existing simle-vicino algorithms, this new approach does not require "blocking." The core maintainer (tfmorris) actively welcomed the contribution and pointed out exactly which backend file (kNNClusterer.java) needs to be modified.

Why I want to work on it: This issue perfectly aligns with my *Domain Continuity* criterion since it directly impacts the backend algorithms of the Clustering feature our team analyzed earlier. It also satisfies the *Community Engagement* and *Appropriate Scope* criteria, as it carries a help wanted label and offers a highly focused algorithmic swap rather than an overarching system rewrite, supported by direct code pointers from the maintainer.

Issue 2: Clustering should be cancelable

Link: <https://github.com/OpenRefine/OpenRefine/issues/289>

Discussion: This issue addresses a performance problem where closing the clustering dialog does not stop the backend computation, causing it to run in the background and consume all CPU resources. The active discussion thread reveals a multi-phase architectural plan by the maintainers to replace legacy dependencies (like the Vicino library) and convert the clustering engine into an asynchronous, cancellable LongRunningProcess complete with a UI progress bar.

Why I want to work on it: This perfectly matches my criteria for *Domain Continuity* and *Community Engagement*. Since I already built mental models for the Clustering feature, this issue provides deep insight into its exact backend limitations. Furthermore, the detailed roadmap discussed by core maintainers (like tfmorris and thadguidry) offers excellent social context, demonstrating how the open-source community collaborates to refactor technical debt and modernize full-stack operations without breaking existing functionality.

Issue 3: Improve UI and UX of the create project tab

Link: <https://github.com/OpenRefine/OpenRefine/issues/5675>

Discussion: This issue focuses on redesigning the "Create Project" tab to reduce cognitive load and improve the flow of information. The community is actively discussing multiple solutions, such as repositioning the "Create Project" action button, reorganizing input fields like "Project Name" and "Tags," and potentially adopting a wizard-style guided dialog.

Why I want to work on it: I selected this issue because it strongly meets the *Community Engagement* and *Appropriate Scope* criteria. Labeled under UX/Usability and outreachy design, it clearly welcomes fresh perspectives from contributors. Working on this would provide an excellent opportunity to dive into OpenRefine's frontend architecture (Module: Frontend) and understand how user inputs are collected and structured before being sent to the backend engine to instantiate a new Project.java model.